



## Laboratorio 03

Clasificación de género y regresión de edad con UTKFace

Dr. Juan Bekios Calfa

**Deep Learning**  
Universidad Católica del Norte

Primer semestre 2026



## Propósito del laboratorio

- Construir un **clasificador de género** a partir de imágenes faciales del dataset **UTKFace**.
- Construir un **regresor de edad** usando las mismas imágenes y el mismo protocolo experimental.
- Comparar distintas estrategias: **baseline clásico, MLP, CNN simple y ResNet preentrenada**.
- Evaluar rendimiento con métricas apropiadas para clasificación y regresión.
- Analizar curvas de entrenamiento, sobreajuste, errores frecuentes y costo computacional.
- Redactar un informe técnico siguiendo la plantilla del laboratorio.



# Objetivos de aprendizaje

1. Preparar un dataset de rostros para entrenamiento con PyTorch.
2. Extraer edad y género desde el nombre de archivo de UTKFace.
3. Implementar modelos con una o dos salidas: clasificación y regresión.
4. Comprender la diferencia entre una línea base clásica, una MLP, una CNN y transferencia de aprendizaje.
5. Comparar resultados con un protocolo justo y reproducible.
6. Presentar tablas, gráficos, ejemplos de errores e interpretación crítica en el informe.



# Problema experimental

## Entrada

Una imagen facial RGB:

$$\mathcal{X} \in \mathbb{R}^{3 \times H \times W}$$

Ejemplo: rostro recortado, alineado y redimensionado a  $224 \times 224$  píxeles.

## Salidas

Clasificación de género:

$$\hat{y}_g \in \{0, 1\}$$

Regresión de edad:

$$\hat{a} \in \mathbb{R}$$

## Idea central

La misma imagen debe alimentar dos tareas: una discreta y otra continua. Por eso se compararán estrategias de una tarea y estrategias multitarea.



# Estrategias a comparar

**E1**

Baseline clásico  
PCA + GaussianNB  
PCA + LinearRegression

**E2**

MLP  
Imagen aplanada  
dos cabezales

**E3**

CNN simple  
Convoluciones  
entrenadas desde cero

**E4**

ResNet  
Transfer learning  
fine-tuning opcional





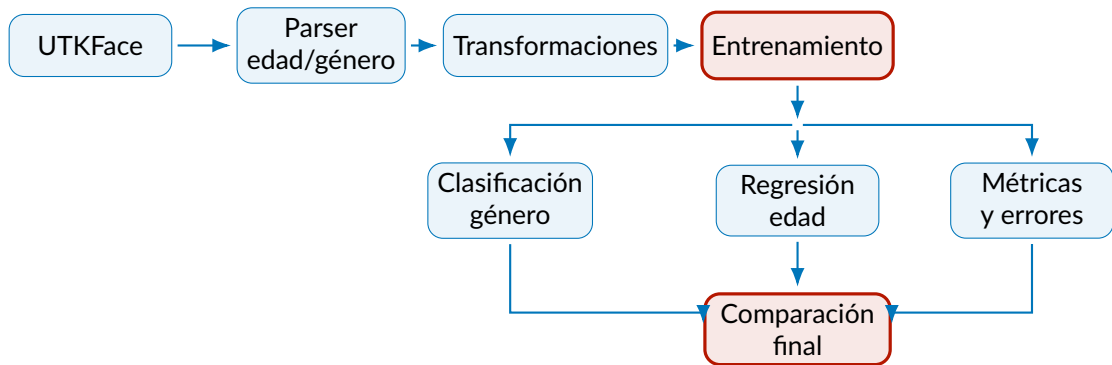
## Criterio de comparación justa

- Usar el mismo conjunto de imágenes y la misma partición entrenamiento/validación/prueba.
- Mantener la misma semilla aleatoria y registrar el entorno de ejecución.
- Aplicar transformaciones equivalentes o justificar transformaciones específicas por arquitectura.
- Evaluar género con métricas de clasificación y edad con métricas de regresión.
- Reportar también costo: parámetros, tiempo por *epoch* y uso de GPU si corresponde.

### Pregunta guía

¿Es suficiente una estrategia simple o se justifica usar una red profunda preentrenada para UTKFace?

## Flujo general del experimento



La comparación final debe incluir métricas, curvas, errores visuales y análisis crítico.



# Estructura del informe: portada y resumen

## Portada

- Título del laboratorio.
- Asignatura, semestre y profesor.
- Integrantes, carrera y correo.

## Resumen

- Problema abordado.
- Dataset UTKFace y tareas.
- Estrategias comparadas.
- Resultado principal y conclusión breve.

## Recomendación

El resumen se escribe al final, pero debe permitir entender el trabajo sin leer todo el informe.





## Estructura del informe: secciones I a III

| Sección           | Debe contener                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------|
| I. Introducción   | Contexto del problema, relevancia de estimar género y edad, objetivos, herramientas usadas y alcance. Máximo una página.      |
| II. Marco teórico | Conceptos de clasificación, regresión, CNN, transferencia de aprendizaje, funciones de pérdida y métricas. Máximo una página. |
| III. Metodología  | Dataset, preprocesamiento, modelos, particiones, hiperparámetros, entrenamiento y evaluación. Máximo dos páginas.             |

## Estructura del informe: secciones IV a VII

| Sección                        | Debe contener                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| IV. Resultados                 | Tablas, gráficos, curvas de entrenamiento, ejemplos visuales y comparación entre estrategias. Máximo tres páginas.         |
| V. Discusión                   | Interpretación de patrones, comparación con lo esperado y explicación de errores. Máximo una página.                       |
| VI. Conclusiones               | Hallazgos clave, limitaciones y mejoras futuras. Máximo una página.                                                        |
| VII. Código y reproducibilidad | Estructura del código, bloques relevantes, ejecución, buenas prácticas y evidencia de reproducibilidad. Máximo una página. |



## Apéndices y autoevaluación

### Apéndice A: Anexos

Puede incluir tablas completas, cálculos intermedios, gráficos adicionales, pruebas exploratorias o fragmentos de código complementarios.

### Apéndice B: Rúbrica

Debe incorporar una autoevaluación: introducción, marco teórico, metodología, resultados, discusión, conclusiones, código, presentación y reproducibilidad.

### Criterio de calidad

El informe no debe ser una colección de capturas. Cada tabla y gráfico debe estar explicado.



## Dataset UTKFace

- **UTKFace** es una base de datos de rostros a gran escala, utilizada frecuentemente en tareas de visión por computador.
- Contiene más de **20.000 imágenes faciales** con anotaciones de **edad, género y grupo étnico**.
- Las imágenes presentan variación en pose, expresión facial, iluminación, oclusión y resolución.
- Para este laboratorio se usará la versión con rostros **recortados y alineados**.
- Esta versión permite concentrarse en el problema de modelado, reduciendo la complejidad asociada a la detección facial.

### Uso en el laboratorio

Se construirán dos tareas usando la misma imagen facial: clasificación de género y regresión de edad.



## Dónde descargar UTKFace

- La página oficial del dataset se encuentra en:

`https://susanqq.github.io/UTKFace/`

- En esa página se debe buscar la sección de descarga asociada a:

`Aligned & Cropped Faces`

- El archivo que se recomienda utilizar para este laboratorio es:

`UTKFace.tar.gz`

- Ese archivo contiene las imágenes faciales ya recortadas y alineadas.
- Después de descargarlo, se debe descomprimir y dejar la carpeta resultante dentro del proyecto.

### Importante

No usar la versión de imágenes completas si el objetivo del laboratorio es comparar modelos sobre rostros ya normalizados espacialmente.



## Formato de nombres en UTKFace

- Las etiquetas no vienen en un archivo CSV separado, sino codificadas directamente en el nombre de cada imagen.

- Una convención habitual de nombre es:

`edad_genero_raza_fecha.jpg`

- Por ejemplo:

`25_1_2_20170116174525125.jpg`

- En este laboratorio se usarán principalmente:

`edad` → variable continua para regresión

`genero` → variable binaria para clasificación

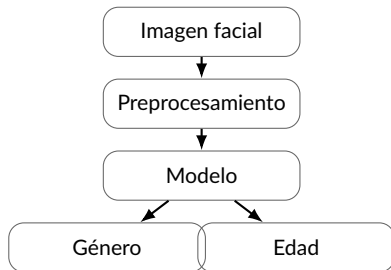
- La variable raza puede analizarse descriptivamente, pero no será el objetivo principal del laboratorio.

## Subconjunto que se utilizará

- Se usarán solo imágenes de la carpeta:

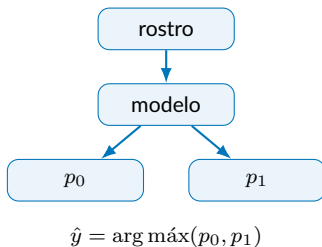
UTKFace/

- Cada imagen debe contener una cara recortada y alineada.
- Las imágenes se cargarán desde disco usando el nombre del archivo para extraer las etiquetas.
- El modelo recibirá como entrada una imagen facial normalizada.
- La salida del sistema tendrá dos componentes:
  - género predicho,
  - edad estimada.



## Tarea 1: clasificación de género

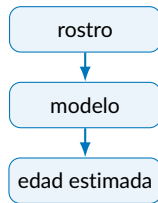
- La salida es discreta.
- El modelo debe estimar la probabilidad de cada clase.
- Se puede entrenar con `CrossEntropyLoss` usando etiquetas enteras.
- Se evalúa con accuracy, precision, recall, F1 y matriz de confusión.





## Tarea 2: regresión de edad

- La salida es continua: una edad estimada.
- Se puede entrenar con `MSELoss` o `SmoothL1Loss`.
- Es esperable que sea más difícil que la clasificación binaria de género.
- Se evalúa con MAE, RMSE y  $R^2$ .



$$\hat{a} \in \mathbb{R}$$



# Preprocesamiento de imágenes

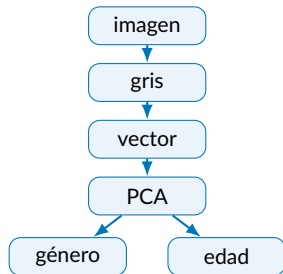
- Redimensionar las imágenes a un tamaño común: por ejemplo  $224 \times 224$  para ResNet.
- Normalizar canales RGB usando medias y desviaciones compatibles con modelos preentrenados.
- Aplicar aumentación de datos solo en entrenamiento: volteo horizontal, recortes o cambios leves de color.
- Para baselines clásicos, también se puede usar una versión en escala de grises y menor resolución.

## Regla metodológica

Las transformaciones de validación y prueba no deben introducir aleatoriedad.

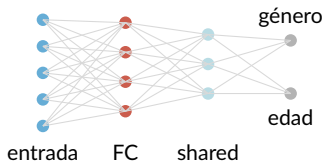
## Estrategia 1: baseline clásico

- Convertir rostro a escala de grises.
- Redimensionar a baja resolución.
- Vectorizar píxeles y aplicar PCA.
- Clasificar género con GaussianNB o LogisticRegression.
- Estimar edad con LinearRegression, Ridge o RandomForestRegressor.



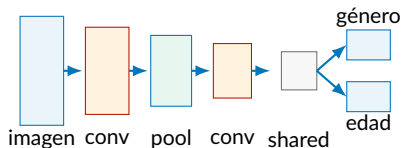
## Estrategia 2: red poco profunda multitarea

- Recibe la imagen aplanada como vector.
- Aprende una representación compartida.
- Tiene dos cabezales: uno para género y otro para edad.
- Sirve como línea base neuronal antes de usar convoluciones.



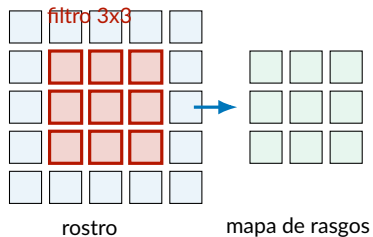
## Estrategia 3: CNN simple multitarea

- Mantiene la estructura espacial de la imagen.
- Aprende filtros locales para rasgos faciales.
- Comparte capas convolucionales para ambas tareas.
- Usa dos cabezales: clasificación y regresión.



# Convolución: idea visual

- Un filtro pequeño se desplaza sobre la imagen.
- En cada posición calcula una combinación local de píxeles.
- Los pesos del filtro se comparten en toda la imagen.
- Esto reduce parámetros y aprovecha patrones faciales locales.





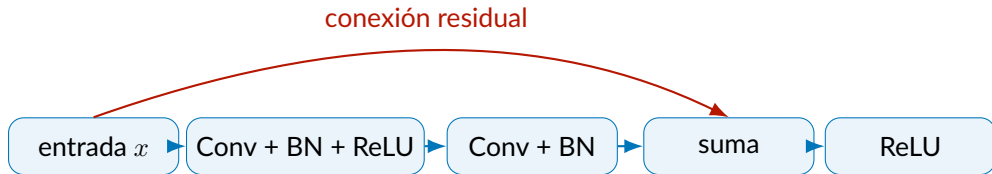
## Estrategia 4: ResNet con transferencia de aprendizaje

- ResNet es una arquitectura profunda con conexiones residuales.
- La red preentrenada ya contiene filtros visuales aprendidos en un dataset grande.
- En este laboratorio se reemplaza la capa final por dos cabezales: género y edad.
- Se puede congelar el extractor de características o ajustar parte de la red.

### Idea clave

Transfer learning permite partir desde una representación visual fuerte, especialmente si se usa una fracción limitada de UTKFace.

## Bloque residual de ResNet



$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$$





# Pérdida combinada para dos tareas

## Clasificación de género

$$\mathcal{L}_g = \text{CrossEntropy}(\hat{y}_g, y_g)$$

## Regresión de edad

$$\mathcal{L}_a = \text{MSE}(\hat{a}, a) \quad \text{o} \quad \mathcal{L}_a = \text{SmoothL1}(\hat{a}, a)$$

## Pérdida total

$$\mathcal{L} = \mathcal{L}_g + \lambda \mathcal{L}_a$$

El parámetro  $\lambda$  controla cuánto pesa la tarea de edad respecto de la tarea de género.



# Métricas de evaluación

## Género

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Matriz de confusión.

## Edad

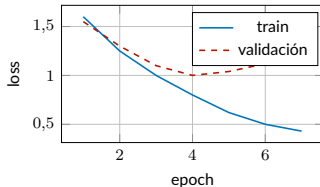
- MAE.
- RMSE.
- $R^2$ .
- Error por rangos etarios.
- Gráfico real vs. predicho.

## Interpretación

Un modelo puede clasificar bien género y aun así estimar mal edad. Las dos tareas deben analizarse por separado.

## Curvas esperadas durante el entrenamiento

- Si train mejora y validation empeora, hay sobreajuste.
- Si ambas curvas son malas, puede faltar capacidad o entrenamiento.
- Si ResNet mejora rápido, puede deberse al preentrenamiento.
- Revisar por separado la pérdida de género y la de edad.



## Tabla de resultados esperada

| Estrategia             | Acc. | F1   | MAE | RMSE | Comentario     |
|------------------------|------|------|-----|------|----------------|
| PCA + modelos clásicos | 0.74 | 0.73 | 9.8 | 13.1 | baseline       |
| MLP multitarea         | 0.78 | 0.77 | 8.9 | 12.0 | simple         |
| CNN simple             | 0.83 | 0.82 | 7.4 | 10.5 | aprende rasgos |
| ResNet multitarea      | 0.88 | 0.87 | 6.1 | 8.9  | transferencia  |

### Importante

Los valores son solo ilustrativos. El informe debe reportar los resultados reales obtenidos por el grupo.



# Paso 1: importar librerías

```
import random
import numpy as np
import pandas as pd
from pathlib import Path
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import transforms, models

from sklearn.metrics import accuracy_score, f1_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import confusion_matrix
```



## Paso 2: fijar semilla y dispositivo

```
def set_seed(seed=42):  
    random.seed(seed)  
    np.random.seed(seed)  
    torch.manual_seed(seed)  
    torch.cuda.manual_seed_all(seed)  
    torch.backends.cudnn.deterministic = True  
    torch.backends.cudnn.benchmark = False  
  
set_seed(42)  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print("Device:", device)
```



## Paso 3: leer etiquetas desde UTKFace

```
def parse_utkface_name(path):  
    """  
    Formato esperado: edad_genero_raza_fecha.jpg  
    Ejemplo: 25_1_2_20170116174525125.jpg  
    """  
    stem = Path(path).stem  
    parts = stem.split("_")  
    age = float(parts[0])  
    gender = int(parts[1])  
    return age, gender  
  
age, gender = parse_utkface_name("25_1_2_20170116174525125.jpg")  
print(age, gender)
```



## Paso 4: transformaciones del dataset

```
train_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

test_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```





## Paso 5: Dataset personalizado

```
class UTKFaceDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.paths = sorted(Path(root_dir).glob("*.jpg"))
        self.transform = transform

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        path = self.paths[idx]
        age, gender = parse_utkface_name(path)
        image = Image.open(path).convert("RGB")
        if self.transform:
            image = self.transform(image)
        return image, torch.tensor(gender).long(), torch.tensor(age).float()
```



## Paso 6: partición y DataLoader

```
dataset = UTKFaceDataset("UTKFace", transform=train_tfms)

n_total = len(dataset)
n_train = int(0.70 * n_total)
n_val = int(0.15 * n_total)
n_test = n_total - n_train - n_val

train_ds, val_ds, test_ds = random_split(
    dataset, [n_train, n_val, n_test],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=32, shuffle=False)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
```



# Modelo 1: MLP multitarea

```
class MultiTaskMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.shared = nn.Sequential(
            nn.Flatten(),
            nn.Linear(3 * 224 * 224, 512), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(512, 128), nn.ReLU()
        )
        self.gender_head = nn.Linear(128, 2)
        self.age_head = nn.Linear(128, 1)

    def forward(self, x):
        h = self.shared(x)
        gender_logits = self.gender_head(h)
        age_pred = self.age_head(h).squeeze(1)
        return gender_logits, age_pred
```



## Modelo 2: CNN simple multitarea

```
class MultiTaskCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.shared = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 28 * 28, 256), nn.ReLU(), nn.Dropout(0.4)
        )
        self.gender_head = nn.Linear(256, 2)
        self.age_head = nn.Linear(256, 1)

    def forward(self, x):
        h = self.shared(self.features(x))
        return self.gender_head(h), self.age_head(h).squeeze(1)
```



## Modelo 3: ResNet multitarea

```
class MultiTaskResNet(nn.Module):
    def __init__(self, freeze_backbone=True):
        super().__init__()
        base = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
        if freeze_backbone:
            for p in base.parameters():
                p.requires_grad = False
        in_features = base.fc.in_features
        base.fc = nn.Identity()
        self.backbone = base
        self.gender_head = nn.Linear(in_features, 2)
        self.age_head = nn.Linear(in_features, 1)

    def forward(self, x):
        h = self.backbone(x)
        return self.gender_head(h), self.age_head(h).squeeze(1)
```



# Entrenamiento de una epoch

```
def train_one_epoch(model, loader, optimizer, lambda_age=0.01):
    model.train()
    ce = nn.CrossEntropyLoss()
    mse = nn.MSELoss()
    total_loss = 0.0

    for images, gender, age in loader:
        images = images.to(device)
        gender = gender.to(device)
        age = age.to(device)
        optimizer.zero_grad()
        gender_logits, age_pred = model(images)
        loss_g = ce(gender_logits, gender)
        loss_a = mse(age_pred, age)
        loss = loss_g + lambda_age * loss_a
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
    return total_loss / len(loader.dataset)
```



# Función de evaluación

```
@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    y_gender, p_gender = [], []
    y_age, p_age = [], []

    for images, gender, age in loader:
        images = images.to(device)
        gender_logits, age_pred = model(images)
        pred_gender = torch.argmax(gender_logits, dim=1).cpu().numpy()
        p_gender.extend(pred_gender)
        y_gender.extend(gender.numpy())
        p_age.extend(age_pred.cpu().numpy())
        y_age.extend(age.numpy())

    acc = accuracy_score(y_gender, p_gender)
    f1 = f1_score(y_gender, p_gender, average="weighted")
    mae = mean_absolute_error(y_age, p_age)
    rmse = mean_squared_error(y_age, p_age, squared=False)
    r2 = r2_score(y_age, p_age)
    return acc, f1, mae, rmse, r2
```



# Entrenar las estrategias neuronales

```
experiments = {  
    "MLP": MultiTaskMLP(),  
    "CNN": MultiTaskCNN(),  
    "ResNet18": MultiTaskResNet(freeze_backbone=True)  
}  
  
results = []  
for name, model in experiments.items():  
    model = model.to(device)  
    optimizer = optim.Adam(  
        filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3  
    )  
    for epoch in range(1, 11):  
        loss = train_one_epoch(model, train_loader, optimizer)  
        val_metrics = evaluate(model, val_loader)  
        print(name, epoch, loss, val_metrics)  
    test_metrics = evaluate(model, test_loader)  
    results.append((name, test_metrics))
```





# Baseline clásico con PCA

```
from sklearn.decomposition import PCA
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.pipeline import Pipeline

# X_train_flat: imagenes vectorizadas de baja resolucion
# y_gender_train: etiquetas de genero
# y_age_train: edades reales

clf_gender = Pipeline([
    ("pca", PCA(n_components=100, whiten=True, random_state=42)),
    ("clf", GaussianNB())
])

reg_age = Pipeline([
    ("pca", PCA(n_components=100, whiten=True, random_state=42)),
    ("reg", Ridge(alpha=1.0))
])

clf_gender.fit(X_train_flat, y_gender_train)
reg_age.fit(X_train_flat, y_age_train)
```

## Diseño experimental sugerido

| Exp. | Estrategia             | Objetivo                                                 |
|------|------------------------|----------------------------------------------------------|
| E1   | PCA + modelos clásicos | Medir una línea base interpretable y rápida.             |
| E2   | MLP multitarea         | Evaluar una red neuronal simple sin convoluciones.       |
| E3   | CNN simple             | Medir el aporte de patrones locales faciales.            |
| E4   | ResNet congelada       | Evaluar transferencia de aprendizaje con bajo costo.     |
| E5   | ResNet fine-tuning     | Medir si ajustar capas profundas mejora ambas tareas.    |
| E6   | Ablación               | Comparar con/sin data augmentation o cambiar $\lambda$ . |



## Qué debe discutirse en resultados

- Número de imágenes usadas y distribución de edades y género.
- Accuracy, F1 y matriz de confusión para género.
- MAE, RMSE,  $R^2$  y gráfico edad real vs. predicha.
- Curvas de pérdida total y, si es posible, pérdida por tarea.
- Tiempo de entrenamiento y número de parámetros entrenables.
- Ejemplos de rostros con predicciones correctas e incorrectas.



# Guía de interpretación

## Si gana el baseline

Puede indicar que el preprocesamiento es fuerte o que las etiquetas se separan por patrones globales. Revisar posible fuga de datos.

## Si gana la CNN simple

Sugiere que los patrones locales faciales ayudan y que el dataset permite aprender filtros desde cero.

## Si gana ResNet

Indica que el preentrenamiento aporta representaciones visuales robustas para ambas tareas.

## Si edad tiene alto error

Analizar rangos etarios, outliers, calidad de etiquetas y si el modelo está priorizando género sobre edad.



## Errores frecuentes en este laboratorio

- Evaluar en test muchas veces mientras se ajustan hiperparámetros.
- Mezclar transformaciones de entrenamiento con transformaciones de prueba.
- No separar claramente métricas de clasificación y regresión.
- Comparar modelos entrenados con *epochs* o particiones distintas sin justificarlo.
- Reportar solo accuracy y omitir el error de edad.
- Pegar todo el código en el informe en vez de explicar bloques relevantes.



## Reproducibilidad: evidencia mínima

- Versión de Python, PyTorch, torchvision, scikit-learn y CUDA.
- Ruta y versión del dataset UTKFace usada.
- Semilla aleatoria y criterio de partición de datos.
- Hiperparámetros: batch size, *epochs*, optimizador, learning rate, weight decay y  $\lambda$ .
- Código modular: datos, modelos, entrenamiento, evaluación y visualización separados.
- Tabla final exportada o registrada para evitar transcripción manual de resultados.



## Mini despliegue con Streamlit

- Además del entrenamiento y evaluación de los modelos, el laboratorio debe incluir un **pequeño despliegue** usando **Streamlit**.
- El objetivo no es construir una aplicación compleja, sino demostrar que el modelo entrenado puede usarse sobre una imagen nueva.
- La aplicación debe permitir cargar una imagen facial y aplicar el mismo preprocesamiento usado durante el entrenamiento.
- Luego debe mostrar dos resultados:
  - **clasificación de género,**
  - **estimación de edad.**
- El despliegue permite conectar el laboratorio con una etapa práctica de uso del modelo.

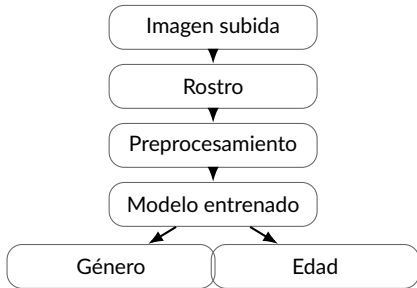
### Idea clave

Un modelo no termina cuando se entrena: también debe poder ejecutarse de forma simple, reproducible y comprensible para un usuario.



## Flujo esperado de la aplicación Streamlit

1. Cargar una imagen desde la interfaz.
2. Mostrar la imagen original.
3. Detectar o recortar el rostro, según corresponda.
4. Aplicar el mismo preprocesamiento usado en entrenamiento.
5. Ejecutar el clasificador de género.
6. Ejecutar el regresor de edad.
7. Mostrar el resultado final de forma clara.

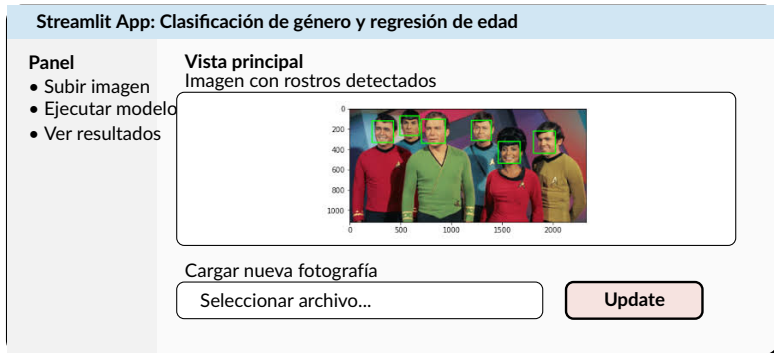




# Prototipo de despliegue en Streamlit

## Ejemplo de interfaz final

Maqueta simple de una aplicación en Streamlit para cargar una imagen, visualizar rostros detectados y actualizar la inferencia.





# Ejemplo mínimo de app en Streamlit

```
import streamlit as st
from PIL import Image
import torch

st.title("Clasificación de género y regresión de edad")
st.write("Laboratorio UTKFace con PyTorch")

uploaded_file = st.file_uploader(
    "Sube una imagen facial",
    type=["jpg", "jpeg", "png"]
)

if uploaded_file is not None:
    image = Image.open(uploaded_file).convert("RGB")
    st.image(image, caption="Imagen cargada", use_container_width=True)

    # 1. Aplicar el mismo preprocesamiento usado en entrenamiento
    x = preprocess_for_inference(image)

    # 2. Ejecutar los modelos entrenados
    gender_pred = gender_model(x)
    age_pred = age_model(x)

    # 3. Mostrar resultados
    st.subheader("Resultado")
    st.write(f"Género predicho: {gender_pred}")
    st.write(f"Edad estimada: {age_pred:.1f} años")
```



## Entregables esperados

- Informe en formato de laboratorio con las secciones solicitadas.
- Código Python o notebook ejecutable.
- Tabla comparativa de estrategias para género y edad.
- Curvas de entrenamiento y validación.
- Matriz de confusión y gráfico edad real vs. predicha.
- Discusión crítica sobre rendimiento, costo y limitaciones.
- Pequeño prototipo de despliegue en **Streamlit** para cargar una imagen nueva y mostrar género predicho y edad estimada.
- Conclusiones con mejoras futuras justificadas.



## Conclusiones del laboratorio

- UTKFace permite estudiar dos tipos de problema con una misma imagen: clasificación y regresión.
- Una línea base clásica ayuda a interpretar cuánto aportan los modelos neuronales.
- Una CNN simple muestra el valor de las convoluciones en rostros.
- Una ResNet preentrenada puede mejorar el rendimiento, pero debe justificarse por métricas y costo.
- El informe debe explicar resultados, no solo mostrarlos.

### Mensaje final

En Deep Learning, entrenar un modelo es solo una parte del trabajo: comparar, interpretar y reproducir el experimento es lo que convierte el resultado en evidencia.

## Referencias sugeridas

-  K. He, X. Zhang, S. Ren y J. Sun,  
*Deep Residual Learning for Image Recognition*,  
CVPR, 2016.
-  Z. Zhang, Y. Song y H. Qi,  
*Age Progression/Regression by Conditional Adversarial Autoencoder*,  
CVPR, 2017. Dataset UTKFace.
-  I. Goodfellow, Y. Bengio y A. Courville,  
*Deep Learning*,  
MIT Press, 2016.
-  J. Bekios-Calfa, J. M. Buenaposada y L. Baumela,  
*Robust Gender Recognition by Exploiting Facial Attributes Dependencies*,  
Pattern Recognition Letters, vol. 36, pp. 228–234, 2014.
-  J. Bekios-Calfa, J. M. Buenaposada y L. Baumela,  
*Class-Conditional Probabilistic Principal Component Analysis: Application to Gender Recognition*,  
Computación y Sistemas, vol. 14, no. 4, pp. 383–391, 2011.
-  J. Bekios-Calfa, J. M. Buenaposada y L. Baumela,  
*Revisiting Linear Discriminant Techniques in Gender Recognition*,  
IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 4, pp. 858–864, 2011.



# Consultas